

7.3.4 节练习

练习 7.32: 定义你自己的 `Screen` 和 `Window_mgr`, 其中 `clear` 是 `Window_mgr` 的成员, 是 `Screen` 的友元。

7.4 类的作用域

每个类都会定义它自己的作用域。在类的作用域之外, 普通的数据和函数成员只能由对象、引用或者指针使用成员访问运算符 (参见 4.6 节, 第 133 页) 来访问。对于类类型成员则使用作用域运算符访问。不论哪种情况, 跟在运算符之后的名字都必须是对应类的成员:

```
Screen::pos ht = 24, wd = 80;           // 使用 Screen 定义的 pos 类型
Screen scr(ht, wd, ' ');
Screen *p = &scr;
char c = scr.get();                     // 访问 scr 对象的 get 成员
c = p->get();                           // 访问 p 所指对象的 get 成员
```

作用域和定义在类外部的成员

一个类就是一个作用域的事实能够很好地解释为什么当我们在类的外部定义成员函数时必须同时提供类名和函数名 (参见 7.1.2 节, 第 230 页)。在类的外部, 成员的名字被隐藏起来了。

一旦遇到了类名, 定义的剩余部分就在类的作用域之内了, 这里的剩余部分包括参数列表和函数体。结果就是, 我们可以直接使用类的其他成员而无须再次授权了。

例如, 我们回顾一下 `Window_mgr` 类的 `clear` 成员 (参见 7.3.4 节, 第 251 页), 该函数的参数用到了 `Window_mgr` 类定义的一种类型:

```
void Window_mgr::clear(ScreenIndex i)
{
    Screen &s = screens[i];
    s.contents = string(s.height * s.width, ' ');
}
```

因为编译器在处理参数列表之前已经明确了我们当前正位于 `Window_mgr` 类的作用域中, 所以不必再专门说明 `ScreenIndex` 是 `Window_mgr` 类定义的。出于同样的原因, 编译器也能知道函数体中用到的 `screens` 也是在 `Window_mgr` 类中定义的。

另一方面, 函数的返回类型通常出现在函数名之前。因此当成员函数定义在类的外部时, 返回类型中使用的名字都位于类的作用域之外。这时, 返回类型必须指明它是哪个类的成员。例如, 我们可能向 `Window_mgr` 类添加一个新的名为 `addScreen` 的函数, 它负责向显示器添加一个新的屏幕。这个成员的返回类型将是 `ScreenIndex`, 用户可以通过它定位到指定的 `Screen`:

```
class Window_mgr {
public:
    // 向窗口添加一个 Screen, 返回它的编号
    ScreenIndex addScreen(const Screen&);
    // 其他成员与之前的版本一致
};
// 首先处理返回类型, 之后我们才进入 Window_mgr 的作用域
```



```
Window_mgr::ScreenIndex
Window_mgr::addScreen(const Screen &s)
{
    screens.push_back(s);
    return screens.size() - 1;
}
```

因为返回类型出现在类名之前，所以事实上它是位于 Window_mgr 类的作用域之外的。在这种情况下，要想使用 ScreenIndex 作为返回类型，我们必须明确指定哪个类定义了它。

7.4 节练习

练习 7.33: 如果我们给 Screen 添加一个如下所示的 size 成员将发生什么情况？如果出现了问题，请尝试修改它。

```
pos Screen::size() const
{
    return height * width;
}
```



7.4.1 名字查找与类的作用域

在目前为止，我们编写的程序中，**名字查找**（name lookup）（寻找与所用名字最匹配的声明的过程）的过程比较直截了当：

- 首先，在名字所在的块中寻找其声明语句，只考虑在名字的使用之前出现的声明。
- 如果没找到，继续查找外层作用域。
- 如果最终没有找到匹配的声明，则程序报错。

284

对于定义在类内部的成员函数来说，解析其中名字的方式与上述的查找规则有所区别，不过在当前的这个例子中体现得不太明显。类的定义分两步处理：

- 首先，编译成员的声明。
- 直到类全部可见后才编译函数体。



编译器处理完类中的全部声明后才会处理成员函数的定义。

按照这种两阶段的方式处理类可以简化类代码的组织方式。因为成员函数体直到整个类可见后才会被处理，所以它能使用类中定义的任何名字。相反，如果函数的定义和成员的声明被同时处理，那么我们将不得不在成员函数中只使用那些已经出现的名字。

用于类成员声明的名字查找

这种两阶段的处理方式只适用于成员函数中使用的名字。声明中使用的名字，包括返回类型或者参数列表中使用的名字，都必须在使用前确保可见。如果某个成员的声明使用了类中尚未出现的名字，则编译器将会在定义该类的作用域中继续查找。例如：

```
typedef double Money;
string bal;
class Account {
public:
```

```

    Money balance() { return bal; }
private:
    Money bal;
    // ...
};

```

当编译器看到 `balance` 函数的声明语句时，它将在 `Account` 类的范围内寻找对 `Money` 的声明。编译器只考虑 `Account` 中在使用 `Money` 前出现的声明，因为没找到匹配的成员，所以编译器会接着到 `Account` 的外层作用域中查找。在这个例子中，编译器会找到 `Money` 的 `typedef` 语句，该类型被用作 `balance` 函数的返回类型以及数据成员 `bal` 的类型。另一方面，`balance` 函数体在整个类可见后才被处理，因此，该函数的 `return` 语句返回名为 `bal` 的成员，而非外层作用域的 `string` 对象。

类型名要特殊处理

一般来说，内层作用域可以重新定义外层作用域中的名字，即使该名字已经在内层作用域中使用过。然而在类中，如果成员使用了外层作用域中的某个名字，而该名字代表一种类型，则类不能在之后重新定义该名字：

285

```

typedef double Money;
class Account {
public:
    Money balance() { return bal; } // 使用外层作用域的 Money
private:
    typedef double Money;          // 错误：不能重新定义 Money
    Money bal;
    // ...
};

```

需要特别注意的是，即使 `Account` 中定义的 `Money` 类型与外层作用域一致，上述代码仍然是错误的。

尽管重新定义类型名字是一种错误的行为，但是编译器并不为此负责。一些编译器仍将顺利通过这样的代码，而忽略代码有错的事实。



类型名的定义通常出现在类的开始处，这样就能确保所有使用该类型的成员都出现在类名的定义之后。

成员定义中的普通块作用域的名字查找

成员函数中使用的名字按照如下方式解析：

- 首先，在成员函数内查找该名字的声明。和前面一样，只有在函数使用之前出现的声明才被考虑。
- 如果在成员函数内没有找到，则在类内继续查找，这时类的所有成员都可以被考虑。
- 如果类内也没找到该名字的声明，在成员函数定义之前的作用域内继续查找。

一般来说，不建议使用其他成员的名字作为某个成员函数的参数。不过为了更好地解释名字的解析过程，我们不妨在 `dummy_fcn` 函数中暂时违反一下这个约定：

```

// 注意：这段代码仅为了说明而用，不是一段很好的代码
// 通常情况下不建议为参数和成员使用同样的名字
int height;                      // 定义了一个名字，稍后将在 Screen 中使用

```

```

class Screen {
public:
    typedef std::string::size_type pos;
    void dummy_fcn(pos height) {
        cursor = width * height;    // 哪个 height? 是那个参数
    }
private:
    pos cursor = 0;
    pos height = 0, width = 0;
};

```

286

当编译器处理 `dummy_fcn` 中的乘法表达式时，它首先在函数作用域内查找表达式中用到的名字。函数的参数位于函数作用域内，因此 `dummy_fcn` 函数体内用到的名字 `height` 指的是参数声明。

在此例中，`height` 参数隐藏了同名的成员。如果想绕开上面的查找规则，应该将代码变为：

```

// 不建议的写法：成员函数中的名字不应该隐藏同名的成员
void Screen::dummy_fcn(pos height) {
    cursor = width * this->height;    // 成员 height
    // 另外一种表示该成员的方式
    cursor = width * Screen::height;  // 成员 height
}

```

Note

尽管类的成员被隐藏了，但我们仍然可以通过加上类的名字或显式地使用 `this` 指针来强制访问成员。

其实最好的确保我们使用 `height` 成员的方法是给参数起个其他名字：

```

// 建议的写法：不要把成员名字作为参数或其他局部变量使用
void Screen::dummy_fcn(pos ht) {
    cursor = width * height;    // 成员 height
}

```

在此例中，当编译器查找名字 `height` 时，显然在 `dummy_fcn` 函数内部是找不到的。编译器接着会在 `Screen` 内查找匹配的声明，即使 `height` 的声明出现在 `dummy_fcn` 使用它之后，编译器也能正确地解析函数使用的是名为 `height` 的成员。

类作用域之后，在外围的作用域中查找

如果编译器在函数和类的作用域中都没有找到名字，它将接着在外围的作用域中查找。在我们的例子中，名字 `height` 定义在外层作用域中，且位于 `Screen` 的定义之前。然而，外层作用域中的对象被名为 `height` 的成员隐藏掉了。因此，如果我们需要的是外层作用域中的名字，可以显式地通过作用域运算符来进行请求：

```

// 不建议的写法：不要隐藏外层作用域中可能被用到的名字
void Screen::dummy_fcn(pos height) {
    cursor = width * ::height;    // 哪个 height? 是那个全局的
}

```

Note

尽管外层的对象被隐藏掉了，但我们仍然可以用作用域运算符访问它。

在文件中名字的出现处对其进行解析

287

当成员定义在类的外部时，名字查找的第三步不仅要考虑类定义之前的全局作用域中的声明，还需要考虑在成员函数定义之前的全局作用域中的声明。例如：

```
int height; // 定义了一个名字，稍后将在 Screen 中使用
class Screen {
public:
    typedef std::string::size_type pos;
    void setHeight(pos);
    pos height = 0; // 隐藏了外层作用域中的 height
};
Screen::pos verify(Screen::pos);
void Screen::setHeight(pos var) {
    // var: 参数
    // height: 类的成员
    // verify: 全局函数
    height = verify(var);
}
```

请注意，全局函数 `verify` 的声明在 `Screen` 类的定义之前是不可见的。然而，名字查找的第三步包括了成员函数出现之前的全局作用域。在此例中，`verify` 的声明位于 `setHeight` 的定义之前，因此可以被正常使用。

7.4.1 节练习

练习 7.34: 如果我们把第 256 页 `Screen` 类的 `pos` 的 `typedef` 放在类的最后一行会发生什么情况？

练习 7.35: 解释下面代码的含义，说明其中的 `Type` 和 `initVal` 分别使用了哪个定义。如果代码存在错误，尝试修改它。

```
typedef string Type;
Type initVal();
class Exercise {
public:
    typedef double Type;
    Type setVal(Type);
    Type initVal();
private:
    int val;
};
Type Exercise::setVal(Type parm) {
    val = parm + initVal();
    return val;
}
```

7.5 构造函数再探

288

对于任何 C++ 的类来说，构造函数都是其中重要的组成部分。我们已经在 7.1.4 节（第 235 页）中介绍了构造函数的基础知识，本节将继续介绍构造函数的一些其他功能，并对